

TUTORIAL GIT/GITHUB

Gestión de versiones

En este laboratorio vamos a aprender a usar GitHub para tener nuestros repositorios de código y vamos a aplicarlo a la creación de una página web estática muy sencilla.

Parte 1 – Uso de Git y GitHub

Git es la herramienta de control de versiones más usada en el mundo. GitHub es la plataforma online más habitual, aunque existen otras como GitLab. Podemos trabajar con el control de versiones tanto por comando (Git) como por interfaz online (GitHub).

1. Una vez instalado Git, podemos comprobar que está bien instalado y ver la versión escribiendo

```
git version
```

en la consola de Git (Git bash) o en la terminal de Mac.

2. Para terminar la configuración de Git, también necesitaremos añadir a la configuración nuestro nombre y correo. Esto nos permitirá conocer quién ha hecho cada cambio en el control de versiones

```
git config --global user.name "mi nombre"  
git config --global user.email "myemail@example.com"
```

3. Para ver la ayuda de Git solo necesitamos usar

```
git help
```

4. En la práctica queremos crear un repositorio donde tengamos o tendremos en el futuro nuestro código. Para ello en esa carpeta hacemos

```
git init
```

Esta línea crea una carpeta oculta llamada .git, al estilo de .vscode. En esta práctica vamos a crear una carpeta “lab” en algún lugar de nuestro ordenador local y vamos a crear un repo allí. Nos mostrará un mensaje como “Initialized empty Git repository in lab/.git/”. Esto significa que todo lo que incluyamos en esta carpeta (otras carpetas, archivos con código, datos, gráficas, documentación...) podremos versionarlo y guardarlo en un repositorio en GitHub.com para que cualquiera (con permisos) pueda descargarlo, modificarlo, etc.

5. Para ver qué archivos podemos versionar, usamos

```
git status
```

De momento no hay nada que guardar (No commits yet, nothing to commit), así que vamos a crear un archivo, por ejemplo un main.cpp en esta carpeta. ¿Cuál es el resultado de hacer git status ahora?

6. Ahora tenemos archivos “untracked”, un archivo que no está en el repositorio. Para añadirlo simplemente seguimos las instrucciones, escribiendo

```
git add main.cpp
```

7. Ahora en el status main.cpp nos aparece en verde porque ya está añadido al “stage”, una zona intermedia antes del guardado final. El repositorio local tiene tres árboles administrados por git: el directorio de trabajo que contiene los archivos, el stage en el que estamos ahora, y el “Head” que afianza esos cambios. En la fase en la que estamos ahora todavía podemos dar marcha atrás y “unstage” los cambios (haciendo git rm... como nos dice la terminal). Para terminar de guardar los cambios tenemos que “comitearlos”, llevarlos por fin al repositorio con un mensaje sobre los cambios que se han hecho, por ejemplo:

```
git commit -m "creacion del main"
```

8. Al ejecutar esta línea nos indica que se ha cambiado 1. Si ejecutamos el log podremos ver todos los cambios que se han hecho en este repositorio, cuándo y quién.

```
git log
```

9. Ahora nuestro repo tiene un archivo main.cpp vacío. Cada vez que modifiquemos ese archivo tendremos que añadir los cambios al repo. En esta práctica vamos a añadir una función main al archivo main.cpp, guardarlo en local y volver a ver el status. ¿Qué pasa ahora con el archivo main?

10. El archivo vuelve a estar en rojo porque se ha modificado y tiene cambios no en stage (en la parte intermedia). Vamos a añadirlos y comitearlos:

```
git add main.cpp  
git commit -m "funcion main"
```

Ahora nos dice que hemos insertado 9 líneas y cambiado un archivo. En el status no hay nada que comitear y el árbol de trabajo está limpio. El log muestra un nuevo commit. La parte que indica (HEAD -> main) es el punto actual.

11. El control de versiones es útil en el caso de que queramos volver hacia atrás. Supongamos que el añadir el último cambio ha “roto” algo en producción, para deshacer ese cambio sólo necesitamos conocer el id del commit que aparece en el log y usar git reset

```
git reset --hard bc7cecdcc9d42e616116f7a4d36fde43314708b5
```

Ahora el log sólo muestra un commit, el inicial, y nuestro archivo main.cpp está vacío.

Hasta ahora hemos trabajado en la rama “main” (On branch main), que es donde suele estar el código estable que se lleva a producción. Para evitar que uno de los cambios llegue al usuario o para evitar conflictos cuando varios desarrolladores trabajan en el mismo proyecto, se suele trabajar en distintas ramas. En general la rama final es main y también hay otra rama “develop” que es sobre la que se trabaja. Cada desarrollador “saca” una rama de develop para añadir su código. Cuando el equipo está convencido que el código está bien, hace su trabajo y no va a romper nada, se junta ese código con el de develop.

12. Para sacar una rama nueva de nuestro proyecto, por ejemplo la rama develop, solo tenemos que usar

```
git branch develop
```

Para ver en qué rama estamos

```
git branch
```

(o git status) y para cambiar de rama

```
git checkout develop
```

De aquí sacamos otra rama, para crear una funcionalidad, por ejemplo “registro”.

```
git branch registro  
git checkout registro
```

Todo el trabajo que realicemos se comiteará en esa rama.

13. Una vez que estamos contentos con nuestro trabajo y estamos en nuestro propio proyecto, juntamos la rama registro con develop usando merge. Vamos a la rama a la que queremos llegar, en este caso develop, mergeamos con registro, y eliminamos registro.

```
git checkout develop  
git merge registro  
git branch -d registro
```

14. Esto solo lo hacemos en nuestro Proyecto porque no necesitamos que nadie más valore nuestro trabajo. En un entorno colaborativo (por ejemplo la práctica de la asignatura, el trabajo de desarrollo en una empresa, o incluso añadir una funcionalidad a un repositorio abierto), necesitamos proponer un cambio y que otros desarrolladores (uno o más, dependiendo de la política del repositorio) lo acepten. Esto se llama un pull request.

15. Como ya tenemos una cuenta de GitHub, podemos versionar nuestro código no solo en local sino también en un repositorio remoto (en la nube) en GitHub. Hay varias maneras de hacerlo, la más sencilla es entrar a nuestra cuenta de GitHub y crear un repositorio nuevo:

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner *  IrinaArevalo /

Great repository names are short and memorable. Need inspiration? How about [symmetrical-fishstick](#) ?

Description (optional)

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: **None** ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: **None** ▾

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

 You are creating a public repository in your personal account.

Create repository

Nuestro nuevo repositorio directamente nos indica qué escribir en nuestro git bash/terminal para comitear los cambios al repo remoto:

Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

En mi caso:

```
git remote add origin https://github.com/IrinaArevalo/lab.git
git push -u origin main
```

 **lab** Private Unwatch 1 Fork 0 Star 0

main 1 Branch Tags Add file Code About

 IrinaArevalo creacion del main bc7cecd · 36 minutes ago 1 Commit

 main.cpp creacion del main 36 minutes ago

No description, website, or topics provided.

Activity 0 stars

16. Desde la interfaz web, si tuviésemos varias ramas, podríamos crear una pull request y aprobarla nosotros mismos. EJERCICIO
17. También desde la interfaz podemos encontrar algún repositorio que nos interese y clonarlo (descargarlo en nuestro ordenador) para ejecutarlo, cambiarlo... y proponer cambios a través de pull requests. Por ejemplo

```
git clone https://github.com/Taha533/Sentiment-Analysis-of-IMDB-Movie-Reviews.git
```

Parte 2 – Creación de una página web sencilla con GitHub pages

Vamos a poner en práctica lo que hemos visto creando una página web estática muy sencilla. GitHub Pages es un servicio de alojamiento u hospedaje web gratuito disponibilizado por GitHub. Para usarlo necesitamos introducir un archivo html en un repositorio público.

18. El primer paso es ir a nuestra cuenta de GitHub y crear un repositorio **público** que se llame “nombredeusuario”.github.io. Por ejemplo: IrinaArevalo.github.io. Es importante que antes de github.io esté el nombre de usuario exacto. También necesitaremos un README.
19. En local clonamos este repositorio en una carpeta usando

```
git clone https://github.com/username/username.github.io
```

20. En esa carpeta añadimos un archivo index.html y lo modificamos con VSCode para añadir cualquier contenido, por ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mi pagina web</title>
  </head>
  <body>
    <h1>Hello world!</h1>
    <p>Bienvenidos a mi pagina web</p>
  </body>
</html>
```

También podéis encontrar un template que os guste, clonarlo, modificarlo y añadirlo a vuestro repositorio. Aquí hay unas cuantas:

<https://github.com/guilyx/awesome-github-pages-portfolios?tab=readme-ov-file>

21. Cuando tengamos un código html que ya queramos ver en nuestra página sólo tenemos que comitear los cambios, añadirlos al repositorio remoto, y después de un rato (cuando la acción en “Actions” se haya completado) podremos ver nuestra página en la url “nombredeusuario”.github.io.